Solution Design Document – Sales Recommendation System

Version History

v1.0 Initial draft

Executive Summary

The objective of this project was to design and build an interactive sales performance dashboard using Streamlit to analyze a USA Superstore dataset.

Purpose of Solution Design Document

This document translates business reporting and analytics requirements into a clear, structured technical solution that can be developed, reviewed, and extended consistently.

Project Background

Built an interactive Streamlit dashboard to convert raw Superstore sales data into actionable insights through dynamic KPIs, filters, and visual analytics.

Project Team

Data Scientist, Data Engineer, Business Analyst

Point of Contact

Hitesh M — Project Developer / Analyst

Business Requirements

Provide an interactive analytics solution that enables business users to analyze sales performance, identify high-performing products and categories through data-driven insights.

Scope

This project delivers an interactive Streamlit dashboard for descriptive analysis of Superstore sales data, providing KPI tracking, filtering, and visual insights while laying the foundation for future advanced analytics and AI-driven features.

Success Criteria

Stakeholders can identify top-performing regions, categories, and products without manual analysis.

Solution Overview

The dashboard enables business users to explore sales performance across time, geography, product categories, and customer segments without requiring technical expertise. It serves as a self-service analytics tool that replaces static reports and manual data analysis.

Automation Flow

1. User opens Streamlit app.
2. App loads environment variables from .env (DB_HOST, DB_NAME, DB_USER, DB_PASS).
3. A cached SQLAlchemy engine is created/reused to connect to MySQL.
4. A cached query pulls data from MySQL table `superstore` into a Pandas DataFrame.
5. Data is cleaned (dates parsed, sales cast to numeric, postal_code normalized).
6. Sidebar filters (date range, region, category, segment) are applied in-memory to the DataFrame.
7. KPIs and charts are recomputed from the filtered data and rendered with Plotly.
8. Users can preview filtered rows and download the filtered dataset as CSV.

Implementation Timeline

- Phase 1 — Data setup: Load Superstore data into MySQL; validate schema and data types.
- Phase 2 — App foundation: Streamlit layout, environment configuration, MySQL connection via SQLAlchemy.
- Phase 3 — Analytics layer: KPI calculations, filter logic, aggregations for category/region/monthly trend.

Technology Stack

Python, Streamlit, Pandas, Plotly Express, MySQL, SQLAlchemy (PyMySQL), python-dotenv

Current State Process Flow

Before this solution, users typically relied on static spreadsheets and manual filtering/aggregation to answer questions such as sales by region/category and top products which was time-consuming and error-prone.

Future State Process Flow

With this solution, sales data is stored centrally in MySQL and surfaced through a self-service Streamlit dashboard. Users filter and explore data interactively; KPIs and visualizations update instantly. The design supports future enhancements such as scheduled ETL refresh, role-based access, and recommendation/forecasting modules.

Functional Requirements

- Provide sidebar filters for Order Date Range, Region, Category, and Segment.
- Display KPI tiles: Total Sales, Orders, Customers, and Average Order Value.
- Show bar charts for Sales by Category and Sales by Region.
- Show a time-series chart for Monthly Sales Trend.
- Show Top 10 Products by Sales.
- Provide a filtered data preview (first 100 rows) and CSV download of filtered data.

Technical Requirements

- MySQL database available and reachable from the Streamlit runtime environment.
- `.env` file present with DB_HOST, DB_PORT , DB_NAME, DB_USER, DB_PASS.
- SQLAlchemy engine cached with `st.cache_resource` to avoid reconnecting on each rerun.
- Data pull cached with `st.cache_data(ttl=60)` to limit database reads while keeping data reasonably fresh.
- Core columns expected: order_date; optional columns used for visuals: sales, category, region, segment, product_name, order_id, customer_id.
- Basic data type coercion for dates, numeric sales, and postal_code string normalization.

Assumptions and Considerations

Sufficient historical data is available in the `superstore` table, and key fields (especially order_date and sales) are populated consistently enough to support time-series and KPI reporting.

Risks and Dependencies

Primary risks include data quality issues (missing/invalid dates, non-numeric sales), schema drift in the MySQL table, database connectivity/credential problems, and changes to business definitions.